

Week 11 — Pygame Examples

Paint series & Sprite / Racer demos

Paint — 01: Mouse Events

```
import pygame

# Mouse events:
#   MOUSEBUTTONDOWN  - button pressed (event.button: 1=LMB, 2=MMB, 3=RMB)
#   MOUSEMOTION      - mouse moved   (event.pos, event.rel)
#   MOUSEBUTTONUP    - button released (event.button)

pygame.init()

WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("01 - Mouse Events")
clock = pygame.time.Clock()

LMBpressed = False
THICKNESS = 5

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            print("LMB pressed!")
            LMBpressed = True

        if event.type == pygame.MOUSEMOTION:
            print(f"pos: {event.pos}  LMB: {LMBpressed}")

        if event.type == pygame.MOUSEBUTTONUP and event.button == 1:
            print("LMB released!")
            LMBpressed = False

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_EQUALS:
                THICKNESS += 1
                print(f"thickness: {THICKNESS}")
            if event.key == pygame.K_MINUS:
                THICKNESS = max(1, THICKNESS - 1)
                print(f"thickness: {THICKNESS}")

    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Paint — 02: Freehand Lines

```
import pygame

# Freehand drawing using smooth lines.
# Problem with dots (rect): gaps appear when mouse moves fast.
# Solution: draw a LINE from (prevX, prevY) to (currX, currY) every frame.

pygame.init()

WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("02 - Freehand Lines")
clock = pygame.time.Clock()

LMBpressed = False
THICKNESS = 5

currX = currY = 0
prevX = prevY = 0

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            LMBpressed = True
            # on first press, prev = curr to avoid line from (0,0)
            prevX, prevY = event.pos
            currX, currY = event.pos

        if event.type == pygame.MOUSEMOTION:
            if LMBpressed:
                currX, currY = event.pos # update current position

        if event.type == pygame.MOUSEBUTTONUP and event.button == 1:
            LMBpressed = False

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_EQUALS:
                THICKNESS += 1
            if event.key == pygame.K_MINUS:
                THICKNESS = max(1, THICKNESS - 1)
            if event.key == pygame.K_c:
                screen.fill("black")

    if LMBpressed:
        pygame.draw.line(screen, "red", (prevX, prevY), (currX, currY), THICKNESS)

    # save current as previous for next frame
    prevX, prevY = currX, currY

    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Paint — 03: Rectangle Tool

```
import pygame

# Rectangle drawing tool.
# Problem: pygame.Rect(x, y, w, h) requires POSITIVE width and height.
# If the user drags up or left, w/h become negative → rect doesn't draw.
# Solution: calculate_rect() always returns a valid rect regardless of drag direction.

pygame.init()

WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("03 - Rectangle Tool")
clock = pygame.time.Clock()

LMBpressed = False
THICKNESS = 3

startX = startY = 0
currX = currY = 0

def calculate_rect(x1, y1, x2, y2):
    """Return a valid Rect from any two corner points (handles negative w/h)."""
    return pygame.Rect(
        min(x1, x2),
        min(y1, y2),
        abs(x1 - x2),
        abs(y1 - y2)
    )

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            LMBpressed = True
            startX, startY = event.pos

        if event.type == pygame.MOUSEMOTION:
            if LMBpressed:
                currX, currY = event.pos
                # draw preview while dragging (leaves trail – see 04.py for fix)
                pygame.draw.rect(
                    screen, "red",
                    calculate_rect(startX, startY, currX, currY),
                    THICKNESS
                )

        if event.type == pygame.MOUSEBUTTONUP and event.button == 1:
            LMBpressed = False
            currX, currY = event.pos
            pygame.draw.rect(
                screen, "red",
                calculate_rect(startX, startY, currX, currY),
                THICKNESS
            )

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_EQUALS:
```

```
        THICKNESS += 1
    if event.key == pygame.K_MINUS:
        THICKNESS = max(1, THICKNESS - 1)
    if event.key == pygame.K_c:
        screen.fill("black")

    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Paint — 04: Rubber-band Preview (base_layer)

```
import pygame

# Rubber-band rectangle preview using a base_layer Surface.
#
# Problem in 03.py: every MOUSEMOTION draws a new rect on top of the last one
# → the screen fills with overlapping rectangles during drag.
#
# Bad fix: screen.fill() before each preview → erases everything the user drew.
#
# Correct fix: base_layer
# - base_layer stores the "committed" canvas (only finished rectangles)
# - on MOUSEMOTION: restore screen from base_layer, then draw the current preview
# - on MOUSEBUTTONUP: draw the final rect, then save screen → base_layer

pygame.init()

WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
base_layer = pygame.Surface((WIDTH, HEIGHT)) # starts black (all zeros)
pygame.display.set_caption("04 - Rubber-band Preview (base_layer)")
clock = pygame.time.Clock()

LMBpressed = False
THICKNESS = 3

startX = startY = 0
currX = currY = 0

def calculate_rect(x1, y1, x2, y2):
    return pygame.Rect(min(x1, x2), min(y1, y2), abs(x1 - x2), abs(y1 - y2))

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            LMBpressed = True
            startX, startY = event.pos

        if event.type == pygame.MOUSEMOTION:
            if LMBpressed:
                currX, currY = event.pos
                # Step 1: restore the committed canvas
                screen.blit(base_layer, (0, 0))
                # Step 2: draw the live preview on top
                pygame.draw.rect(
                    screen, "red",
                    calculate_rect(startX, startY, currX, currY),
                    THICKNESS
                )

        if event.type == pygame.MOUSEBUTTONUP and event.button == 1:
            LMBpressed = False
            currX, currY = event.pos
            # Draw the final rectangle
            pygame.draw.rect(
                screen, "red",
                calculate_rect(startX, startY, currX, currY),
```

```
        THICKNESS
    )
    # Commit: save the current screen into base_layer
    base_layer.blit(screen, (0, 0))

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_EQUALS:
            THICKNESS += 1
        if event.key == pygame.K_MINUS:
            THICKNESS = max(1, THICKNESS - 1)
        if event.key == pygame.K_c:
            screen.fill("black")
            base_layer.fill("black")    # clear both layers!

    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Sprites — 01: Basic Sprite

```
import pygame

# A Sprite = game object with two required attributes:
# .image — the Surface that gets drawn
# .rect — the position / bounding box on screen

pygame.init()

WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("01 - Basic Sprite")
clock = pygame.time.Clock()

class Ball(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__() # required
        self.image = pygame.Surface((50, 50))
        self.image.fill((255, 50, 50))
        self.rect = self.image.get_rect(center=(WIDTH // 2, HEIGHT // 2))

    def update(self):
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]: self.rect.move_ip(-5, 0)
        if keys[pygame.K_RIGHT]: self.rect.move_ip(5, 0)
        if keys[pygame.K_UP]: self.rect.move_ip(0, -5)
        if keys[pygame.K_DOWN]: self.rect.move_ip(0, 5)

ball = Ball()

all_sprites = pygame.sprite.Group()
all_sprites.add(ball)

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    all_sprites.update() # calls update() on every sprite in the group

    screen.fill("black")
    all_sprites.draw(screen) # draws every sprite using .image at .rect position
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```


Sprites — 02: Sprite Groups

```
import pygame
import random

# Sprite Groups let you manage, update, and draw many sprites at once.
# group.update() → calls update() on every member
# group.draw(surface) → blits every member's .image at its .rect

pygame.init()

WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("02 - Sprite Groups")
clock = pygame.time.Clock()

class Star(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        size = random.randint(4, 12)
        self.image = pygame.Surface((size, size))
        self.image.fill("white")
        self.rect = self.image.get_rect()
        self.rect.x = random.randint(0, WIDTH)
        self.rect.y = random.randint(0, HEIGHT)
        self.speed = random.randint(1, 4)

    def update(self):
        self.rect.y += self.speed
        if self.rect.top > HEIGHT:          # wrap: reappear at the top
            self.rect.bottom = 0
            self.rect.x = random.randint(0, WIDTH)

stars = pygame.sprite.Group()
for _ in range(60):
    stars.add(Star())

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    stars.update()
    screen.fill("black")
    stars.draw(screen)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Sprites — 03: Player and Enemy

```
import pygame
import random

# Player controlled with arrow keys.
# Enemy falls from the top, resets when it goes off screen.
# move_ip(dx, dy) – moves the rect IN PLACE (no new rect created)

pygame.init()

WIDTH, HEIGHT = 400, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("03 - Player and Enemy")
clock = pygame.time.Clock()
FPS = 60

class Player(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.Surface((50, 30))
        self.image.fill((50, 200, 50))
        self.rect = self.image.get_rect(center=(WIDTH // 2, HEIGHT - 40))
        self.speed = 6

    def update(self):
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            self.rect.move_ip(-self.speed, 0)
        if keys[pygame.K_RIGHT]:
            self.rect.move_ip(self.speed, 0)
        # Keep inside screen
        if self.rect.left < 0:
            self.rect.left = 0
        if self.rect.right > WIDTH:
            self.rect.right = WIDTH

class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.Surface((40, 40))
        self.image.fill((200, 50, 50))
        self.rect = self.image.get_rect()
        self.speed = 5
        self._reset()

    def _reset(self):
        self.rect.left = random.randint(0, WIDTH - self.rect.width)
        self.rect.bottom = 0

    def update(self):
        self.rect.move_ip(0, self.speed)
        if self.rect.top > HEIGHT:
            self._reset()

player = Player()
enemy = Enemy()

all_sprites = pygame.sprite.Group(player, enemy)

running = True
```

```
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    all_sprites.update()

    screen.fill((30, 30, 30))
    all_sprites.draw(screen)
    pygame.display.flip()
    clock.tick(FPS)

pygame.quit()
```

Sprites — 04: Collision Detection

```
import pygame
import random

# Collision detection:
#   spritecollideany(sprite, group) → first colliding sprite or None
#   spritecollide(sprite, group, dokill) → list of colliding sprites
#                                     dokill=True removes them from group

pygame.init()

WIDTH, HEIGHT = 700, 400
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("04 - Collision Detection")
clock = pygame.time.Clock()
font = pygame.font.SysFont("Verdana", 24)

class Block(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.Surface((20, 15))
        self.image.fill("black")
        self.rect = self.image.get_rect()
        self.reset_position()

    def reset_position(self):
        self.rect.y = random.randrange(-300, -20)
        self.rect.x = random.randrange(WIDTH - 20)

    def update(self):
        self.rect.y += 3
        if self.rect.top > HEIGHT:
            self.reset_position()

class Player(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.Surface((20, 15))
        self.image.fill("red")
        self.rect = self.image.get_rect()

    def update(self):
        # Mouse-controlled: follow the cursor
        pos = pygame.mouse.get_pos()
        self.rect.x = pos[0]
        self.rect.y = pos[1]

block_group = pygame.sprite.Group()
for _ in range(50):
    b = Block()
    b.rect.x = random.randrange(WIDTH)
    b.rect.y = random.randrange(HEIGHT)
    block_group.add(b)

player = Player()
all_sprites = pygame.sprite.Group(block_group, player)

score = 0
running = True
while running:
```

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False

all_sprites.update()

# spritecollide returns a list of blocks the player overlaps.
# dokill=False → blocks are NOT removed from group automatically.
hit_list = pygame.sprite.spritecollide(player, block_group, False)
for block in hit_list:
    block.reset_position()
    score += 1

screen.fill("white")
all_sprites.draw(screen)

score_text = font.render(f"Score: {score}", True, "black")
screen.blit(score_text, (10, 10))

pygame.display.flip()
clock.tick(60)

pygame.quit()
```

Sprites — 05: USEREVENT + kill()

```
import pygame
import random

# USEREVENT — custom timed events.
# pygame.time.set_timer(event_type, interval_ms) fires that event every interval.
# sprite.kill() removes the sprite from ALL groups it belongs to.

pygame.init()

WIDTH, HEIGHT = 400, 400
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("05 - USEREVENT + kill()")
clock = pygame.time.Clock()

SPAWN_CAR = pygame.USEREVENT + 1
pygame.time.set_timer(SPAWN_CAR, 1500) # fire every 1.5 seconds

COLORS = ["red", "blue", "orange", "purple", "cyan"]

class Car(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.Surface((30, 50))
        self.image.fill(random.choice(COLORS))
        self.rect = self.image.get_rect()
        self.rect.centerx = random.randint(15, WIDTH - 15)
        self.rect.bottom = 0
        self.speed = random.randint(2, 5)

    def update(self):
        self.rect.y += self.speed
        if self.rect.top > HEIGHT:
            self.kill() # removes from ALL groups; Python garbage-collects it

cars = pygame.sprite.Group()

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        if event.type == SPAWN_CAR:
            cars.add(Car())

    cars.update()

    screen.fill((50, 50, 50))
    cars.draw(screen)

    count = pygame.font.SysFont("Verdana", 20).render(
        f"Cars: {len(cars)}", True, "white"
    )
    screen.blit(count, (10, 10))

    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Racer — Full Game Demo

[illegible]

[illegible]